

Section 7

Task 5

DECODER, ENCODERS AND ONEHOTLOGIC

21-07-2026

PLAYGROUND LINK: <https://www.edaplayground.com/x/ijQn>

GIT LINK: https://github.com/Nooorrr66/MEDS_2026/tree/main/module_4/7_Decoders_Encoders_Onehotlogic

CODE:

As the module mentioned:

- First design and verify the 2-to-4 decoder (with an enable input) as a standalone module.

Test Bench

```
module decoder2to4_tb;
logic en;
logic [1:0] sel;
logic [3:0] y;

decoder2to4 dut(
    .en(en),
    .sel(sel),
    .y(y)
);

initial begin
    en = 0;
    for(int i=0;i<4;i++) begin
        sel=i;
        #10;
        $display("en=%b sel=%b y=%b",en,sel,y);
    end

    en = 1;

    for(int i=0;i<4;i++) begin
        sel=i;
        #10;
        $display("en=%b sel=%b y=%b",en,sel,y);
    end

    $finish;
end
endmodule
```

Design:

```
module decoder2to4(
    input logic en,
    input logic [1:0] sel,
    output logic [3:0] y
);

always_comb begin
    if (!en)
        y = 4'b0000;
    else begin
        case(sel)
            2'b00: y = 4'b0001;
            2'b01: y = 4'b0010;
            2'b10: y = 4'b0100;
            2'b11: y = 4'b1000;
            default: y = 4'b0000;
        endcase
    end
end
```

TERMINAL:

```
Log Share
./simv up to date
CPU time: .550 seconds to compile + .483 seconds to elab + .523 seconds to link
Chronologic VCS simulator copyright 1991-2025
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP1_Full164; Runtime version X-2025.06-SP1_Full164; Jul 21 04:29 2026
en=0 sel=00 y=0000
en=0 sel=01 y=0000
en=0 sel=10 y=0000
en=0 sel=11 y=0000
en=1 sel=00 y=0001
en=1 sel=01 y=0010
en=1 sel=10 y=0100
en=1 sel=11 y=1000
$finish called from file "testbench.sv", line 30.
$finish at simulation time 80
VCS Simulation Report
```

Now as per mentioned:

- Then instantiate it twice and add the glue logic that produces the full 3-to-8 decode.
- Testbench must exhaustively check all 8 select values with enable asserted, plus confirm all outputs are 0 when enable is deasserted.

```
module decoder3to8_tb;

logic en;
logic [2:0] sel;
logic [7:0] y;

decoder3to8 dut (
    .en(en),
    .sel(sel),
    .y(y)
);

initial begin
    // Waveform dump
    $dumpfile("decoder3to8.vcd");
    $dumpvars(0, decoder3to8_tb);

    // Test 1: Decoder disabled

    en = 0;

    $display("==== TEST 1: ENABLE = 0 =====");

    for (int i = 0; i < 8; i++) begin
        sel = i;
        #10;

        if (y !== 8'b00000000)
            $display("[FAIL] en=%b sel=%0d y=%b (Expected 00000000)", en, sel, y);
        else
            $display("[PASS] en=%b sel=%0d y=%b", en, sel, y);
        end

    // Test 2: Decoder enabled

    en = 1;

    $display("\n==== TEST 2: ENABLE = 1 =====");

    for (int i = 0; i < 8; i++) begin
        sel = i;
        #10;
        if (y !== (8'b00000001 << i))
            $display("[FAIL] en=%b sel=%0d y=%b Expected=%b",
                en, sel, y, (8'b00000001 << i));
        else
            $display("[PASS] en=%b sel=%0d y=%b",
                en, sel, y);
        end

    $display("\n==== ALL TESTS COMPLETED =====");

    #10;
    $finish;
end

endmodule
```

```
module decoder2to4(
    input logic en,
    input logic [1:0] sel,
    output logic [3:0] y
);

always_comb begin
    if (!en)
        y = 4'b0000;
    else begin
        case(sel)
            2'b00: y = 4'b0001;
            2'b01: y = 4'b0010;
            2'b10: y = 4'b0100;
            2'b11: y = 4'b1000;
            default: y = 4'b0000;
        endcase
    end
end

endmodule

module decoder3to8(
    input logic en,
    input logic [2:0] sel,
    output logic [7:0] y
);

logic [3:0] low;
logic [3:0] high;

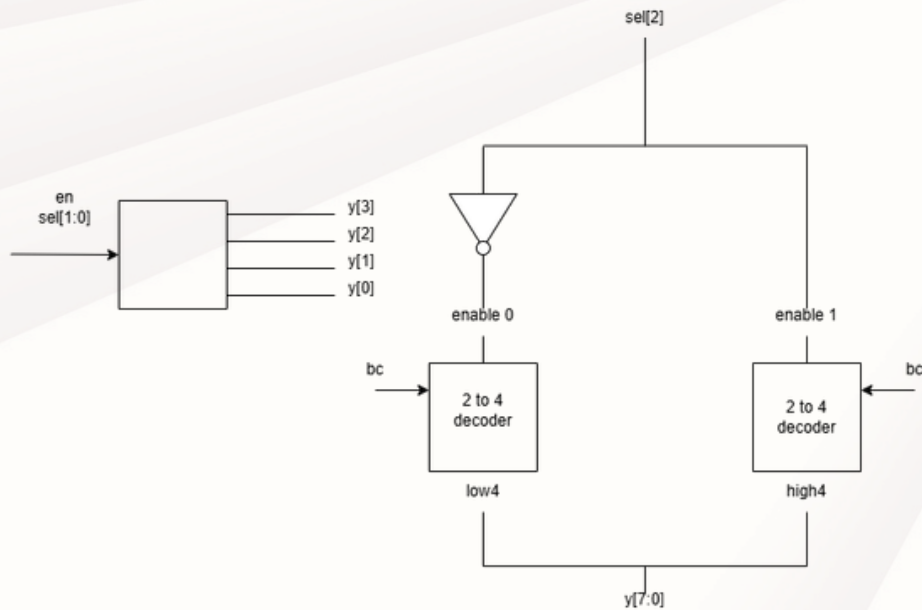
decoder2to4 d0(
    .en(en & ~sel[2]),
    .sel(sel[1:0]),
    .y(low)
);

decoder2to4 d1(
    .en(en & sel[2]),
    .sel(sel[1:0]),
    .y(high)
);

assign y = {high, low};

endmodule
```

TERMINAL:



TERMINAL:

```

@Log      <Share
../simv up to date
CPU time: .632 seconds to compile + .558 seconds to elab + .563 seconds to link
Chronologic VCS simulator copyright 1991-2025
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP1_Full64; Runtime version X-2025.06-SP1_Full64;  Jul 21 05:01 2026
===== TEST 1 : ENABLE = 0 =====
[PASS] en=0 sel=0 y=00000000
[PASS] en=0 sel=1 y=00000000
[PASS] en=0 sel=2 y=00000000
[PASS] en=0 sel=3 y=00000000
[PASS] en=0 sel=4 y=00000000
[PASS] en=0 sel=5 y=00000000
[PASS] en=0 sel=6 y=00000000
[PASS] en=0 sel=7 y=00000000

===== TEST 2 : ENABLE = 1 =====
[PASS] en=1 sel=0 y=00000001
[PASS] en=1 sel=1 y=00000010
[PASS] en=1 sel=2 y=00000100
[PASS] en=1 sel=3 y=00001000
[PASS] en=1 sel=4 y=00010000
[PASS] en=1 sel=5 y=00100000
[PASS] en=1 sel=6 y=01000000
[PASS] en=1 sel=7 y=10000000

===== ALL TESTS COMPLETED =====
$finish called from file "testbench.sv", line 59.

```

WAVEFORM:



EXPLANATION:

The design implements a 3-to-8 decoder hierarchically using two 2-to-4 decoder modules. The most significant select bit ($\text{sel}[2]$) determines which 2-to-4 decoder is enabled. When $\text{sel}[2] = 0$, the lower decoder is enabled, and when $\text{sel}[2] = 1$, the upper decoder is enabled. Both decoders receive the lower two select bits ($\text{sel}[1:0]$) as inputs. Since only one decoder is enabled at a time, only one output bit is asserted, producing a one-hot 8-bit output. The outputs of the two decoders are concatenated to form the final output vector $y[7:0]$.

The testbench exhaustively tests all eight possible select values with the decoder enabled and verifies that the correct one-hot output is generated. It also checks that when the enable signal is deasserted, all outputs remain 0. PASS/FAIL messages are displayed for every test case, and a waveform is generated for visual verification.

The waveform confirms the correct operation of the decoder. During the first half of the simulation, en is held low while sel cycles through all eight input combinations. As expected, the output y remains 00000000 for every select value, showing that the decoder is properly disabled.

In the second half of the simulation, en is asserted high. As sel changes from 000 to 111, the output changes sequentially from 00000001 to 10000000. At every instant, exactly one output bit is high, demonstrating the expected one-hot decoding behavior.

The terminal output displays a PASS message for every test case. When the decoder is disabled ($\text{en} = 0$), every output is correctly reported as 00000000. When the decoder is enabled ($\text{en} = 1$), the output matches the expected one-hot value for each select input. Since all test cases pass, the implementation correctly satisfies the functional requirements of a 3-to-8 decoder.

Task 7

21-07-2026

PLAYGROUND LINK:

<https://www.edaplayground.com/x/kJz4>

CODE:

Test Bench

```
module lzc32_tb;
logic [31:0] in;
logic [5:0] count;
// Device Under Test
lzc32 dut(
    .in(in),
    .count(count)
);
// Waveform

initial begin
    $dumpfile("lzc32.vcd");
    $dumpvars(0, lzc32_tb);
end
// Reference Model
function automatic [5:0] reference_lzc(input logic
[31:0] x);
    reference_lzc = 32;
    for (int i = 31; i >= 0; i--) begin
        if ((reference_lzc == 32) && x[i])
            reference_lzc = 31 - i;
    end
endfunction
// Test Procedure
initial begin
    logic [5:0] expected;
    // Directed Test 1
    in = 32'h00000000;
    #10;

    expected = reference_lzc(in);

    if(count == expected)
        $display("[PASS] Input=%h Count=%0d", in,
count);
    else
        $display("[FAIL] Input=%h Count=%0d
Expected=%0d", in, count, expected);
    // Directed Test 2

    in = 32'h80000000;
    #10;

    expected = reference_lzc(in);

    if(count == expected)
        $display("[PASS] Input=%h Count=%0d", in,
count);
    else
        $display("[FAIL] Input=%h Count=%0d
Expected=%0d", in, count, expected);
    // Directed Test 3
    in = 32'h00008000;
    #10;

    expected = reference_lzc(in);
```

```
    if(count == expected)
        $display("[PASS] Input=%h Count=%0d", in, count);
    else
        $display("[FAIL] Input=%h Count=%0d
Expected=%0d", in, count, expected);
    // Directed Test 4
    in = 32'h00000001;
    #10;

    expected = reference_lzc(in);

    if(count == expected)
        $display("[PASS] Input=%h Count=%0d", in, count);
    else
        $display("[FAIL] Input=%h Count=%0d
Expected=%0d", in, count, expected);
    // 50 Random Tests
    $display("\nRunning Random Tests...\n");

    for(int i=0;i<50;i++) begin
        in = $urandom;
        #10;

        expected = reference_lzc(in);

        if(count == expected)
            $display("[PASS] Input=%h Count=%0d", in, count);
        else
            $display("[FAIL] Input=%h Count=%0d
Expected=%0d", in, count, expected);

    end

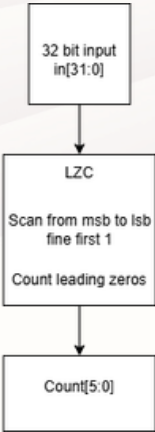
    $display("\nAll tests completed.");
    #10;
    $finish;

end
endmodule
```

Design:

```
module lzc32(
    input logic [31:0] in,
    output logic [5:0] count
);
always_comb begin
    count = 6'd32;
    for (int i = 31; i >= 0; i--) begin
        if ((count == 32) && in[i])
            count = 31 - i;
    end
end
endmodule
```


SCHEMATIC:



TERMINAL:

```
Log Share
Chronologic VCS Simulator Copyright 1991-2024
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP1_Full164; Runtime
[PASS] Input=00000000 Count=32
[PASS] Input=80000000 Count=0
[PASS] Input=00008000 Count=16
[PASS] Input=00000001 Count=31

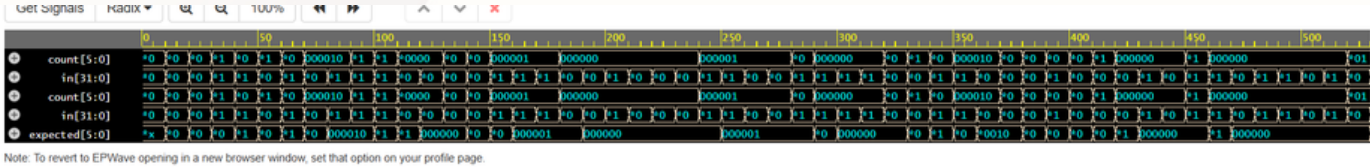
Running Random Tests...

[PASS] Input=3b29e213 Count=2
[PASS] Input=69262e70 Count=1
[PASS] Input=fe9c33fd Count=0
[PASS] Input=3d5d03e2 Count=2
[PASS] Input=20adb297 Count=2
[PASS] Input=0564cbf1 Count=5
[PASS] Input=7a0900c5 Count=1
[PASS] Input=cea5c6ec Count=0
[PASS] Input=f6a47a48 Count=0
[PASS] Input=0947d40c Count=4
[PASS] Input=214b2b2c Count=2
[PASS] Input=70a8386b Count=1
[PASS] Input=51ab091b Count=1
[PASS] Input=73a72e45 Count=1
[PASS] Input=a4d00ef4 Count=0
[PASS] Input=c333cd6c Count=0
[PASS] Input=edca9767 Count=0
[PASS] Input=ad22848c Count=0
[PASS] Input=f1305a4a Count=0
```

```
Log Share
[PASS] Input=f1305a4a Count=0
[PASS] Input=d4e2dca6 Count=0
[PASS] Input=5a239ea3 Count=1
[PASS] Input=6bf0a09d Count=1
[PASS] Input=74a9477c Count=1
[PASS] Input=427eb1b8 Count=1
[PASS] Input=33f729eb Count=2
[PASS] Input=fcbfe75b Count=0
[PASS] Input=b1c059f3 Count=0
[PASS] Input=8bf5064d Count=0
[PASS] Input=0f9ca55c Count=4
[PASS] Input=115c25f6 Count=3
[PASS] Input=93010ad9 Count=0
[PASS] Input=3a360ca9 Count=2
[PASS] Input=394b47aa Count=2
[PASS] Input=08e9b69e Count=4
[PASS] Input=2d582cc8 Count=2
[PASS] Input=de202cad Count=0
[PASS] Input=0c389026 Count=4
[PASS] Input=7bde57f6 Count=1
[PASS] Input=b6a25160 Count=0
[PASS] Input=96b4915b Count=0
[PASS] Input=c823e842 Count=0
[PASS] Input=522dd321 Count=1
[PASS] Input=ef3da191 Count=0
[PASS] Input=9105a738 Count=0
[PASS] Input=efde4d4f Count=0
[PASS] Input=88ec5d6d Count=0
[PASS] Input=d2cc89c2 Count=0
[PASS] Input=c9d70e79 Count=0
[PASS] Input=602b5e7c Count=1
[PASS] Input=5eaf4f2e Count=1

All tests completed.
$finish called from file "testbench.sv", line 87.
```

WAVEFORM:



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

EXPLANATION:

The 32-bit Leading Zero Counter (LZC) counts the number of consecutive zeros starting from the most significant bit until the first logic 1 is encountered. The design uses a combinational `always_comb` block containing a for loop that scans the input from bit 31 down to bit 0. The output is initialized to 32 to handle the special case where the input is all zeros. When the first logic 1 is found, the number of leading zeros is calculated as $31 - i$, where i is the position of the first set bit. The design is fully combinational and produces the output without requiring a clock.

The waveform verifies the correct operation of the Leading Zero Counter. The input vector changes through several directed test cases followed by randomly generated values. For every input, the DUT output (count) matches the reference model output (expected). The waveform confirms that the module correctly counts the number of leading zeros, including the special case where the input is all zeros, producing an output of 32.

A reference function (`reference_lzc`) is included in the testbench. It independently calculates the expected number of leading zeros by scanning the input from the most significant bit to the least significant bit. This serves as the golden reference against which the DUT (Design Under Test) is verified.

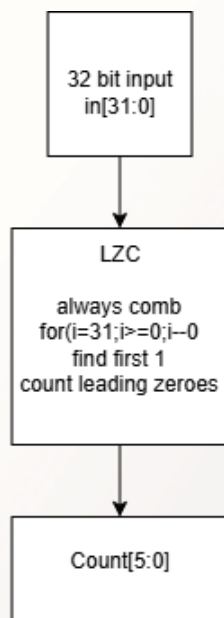
The testbench first performs directed tests using important corner cases:

- `32'h00000000` (all zeros → expected count = 32)
- `32'h80000000` (MSB set → expected count = 0)
- `32'h00008000` (bit 15 set → expected count = 16)
- `32'h00000001` (LSB set → expected count = 31)

After the directed tests, the testbench generates 50 random 32-bit input vectors using `$urandom`. For each input, the expected result is calculated by the reference model and compared with the DUT output. The testbench prints PASS if both values match and FAIL otherwise.

TREE STYLE IMPLEMENTATION:

```
module lzc32_tree(  
    input logic [31:0] in,  
    output logic [5:0] count  
);  
  
always_comb begin  
    logic [31:0] x;  
    x = in;  
    count = 0;  
  
    if (x == 32'b0) begin  
        count = 32;  
    end  
    else begin  
        if (x[31:16] == 16'b0) begin  
            count = count + 16;  
            x = x << 16;  
        end  
  
        if (x[31:24] == 8'b0) begin  
            count = count + 8;  
            x = x << 8;  
        end  
  
        if (x[31:28] == 4'b0) begin  
            count = count + 4;  
            x = x << 4;  
        end  
  
        if (x[31:30] == 2'b0) begin  
            count = count + 2;  
            x = x << 2;  
        end  
  
        if (x[31] == 1'b0)  
            count = count + 1;  
    end  
end  
  
endmodule
```



The above implemented design uses a linear search that checks one bit at a time from the most significant bit toward the least significant bit. In the worst case (when the first set bit is near the LSB or the input is all zeros), every bit must be examined, resulting in a critical path proportional to the input width.

A faster implementation uses a tree-structured approach like this one . Instead of checking one bit at a time, the input is divided into smaller groups (2-bit groups, then 4-bit groups, then 8-bit groups, and so on). Each stage determines whether its group contains a logic 1, and the results are combined in parallel until the position of the first set bit is found. This significantly reduces the logic depth because multiple comparisons occur simultaneously .The linear implementation examines bits sequentially, so the delay increases with the input width. In contrast, the tree implementation performs comparisons in parallel and combines the results over only a few stages. As the input width grows, the tree's critical path grows logarithmically rather than linearly, making it much faster for larger bit widths.